Pulsejet Vortex Flame-Holder Model

MATLAB source listing: pulsejet_vortex_flameholder.m

434 lines | Toroidal vortex + progress-variable flame + Damkohler number evolution

Domain: 0.3 m x 0.24 m chamber (half domain, 80 x 60 grid), explicit time stepping, 0.1 s nominal
Outputs: live 12-panel animation, stability plots, PNG figure and MAT data file
Requires MATLAB R2018b or newer (uses caxis, yyaxis, pcolor, streamline).

Source reproduced verbatim from the author's file. See the README and THEORY PDFs for usage, assumptions and equation

pulsejet_vortex_flameholder.m

```
%% ========================================================================
% TOROIDAL VORTEX FORMATION & FLAME STABILIZATION IN VALVELESS PULSEJET
% Shows vortex formation, flame holding, and Damköhler number evolution
% ========================================================================

function pulsejet_vortex_flameholder()
clear; close all; clc;

%% ==================== 1. PHYSICAL PARAMETERS ==========================
fprintf('=== PULSEJET VORTEX FLAME HOLDER SIMULATION ===\n\n');

% Chamber geometry (from paper optimal ratios)
L_chamber = 0.3;      % Chamber length [m]
D_chamber = 0.24;     % Chamber diameter [m] - L/D = 1.25
R_chamber = D_chamber/2;

% Vortex parameters
D_vortex = 0.6 * D_chamber;    % Vortex diameter = 60% of chamber
R_vortex = D_vortex/2;
L_vortex = 0.3 * L_chamber;

% Fluid properties
rho = 1.2;            % Air density [kg/m³]
mu = 1.8e-5;          % Dynamic viscosity [Pa·s]
nu = mu/rho;          % Kinematic viscosity [m²/s]
cp = 1005;            % Specific heat [J/kg·K]
k_thermal = 0.026;    % Thermal conductivity [W/m·K]

% Combustion properties
T_fresh = 300;        % Fresh mixture temperature [K]
T_flame = 2200;       % Flame temperature [K]
phi = 1.0;            % Equivalence ratio
S_L = 0.4;            % Laminar flame speed [m/s]

% Flow conditions
U_mean = 50;          % Mean flow velocity [m/s]
U_exit = 150;         % Exit velocity [m/s]

% Time parameters
dt = 1e-4;            % Time step [s]
T_total = 0.1;        % Total simulation time [s]
Nt = round(T_total/dt);

%% ==================== 2. COMPUTATIONAL DOMAIN =========================
Nx = 80;              % Points in axial direction
Ny = 60;              % Points in radial direction

% Grid
x = linspace(0, L_chamber, Nx);
y = linspace(0, D_chamber/2, Ny);  % Half domain (axisymmetric)
dx = x(2) - x(1);
dy = y(2) - y(1);
[X, Y] = meshgrid(x, y);

% Vortex center position
x_vortex = L_chamber * 0.4;    % 40% from inlet
y_vortex = D_chamber/4;        % Quarter radius

%% ==================== 3. INITIAL CONDITIONS ===========================
% Base flow field
U = U_mean * (1 - (Y/(D_chamber/2)).^2);
V = zeros(size(U));

% Temperature field
T = T_fresh * ones(size(U));

% Progress variable (c=0: fresh, c=1: burned)
c = zeros(size(U));

% Vorticity field
omega = zeros(size(U));

% Reaction rate
A = 1e8;          % Pre-exponential factor [1/s]
Ea = 1.5e5;       % Activation energy [J/mol]
R_gas = 8.314;    % Gas constant [J/mol·K]

%% ==================== 4. VORTEX FORMATION MODEL =======================
fprintf('Vortex formation model initialization...\n');
fprintf('Vortex diameter: %.3f m (%.1f%% of chamber)\n', D_vortex, D_vortex/D_chamber*100);

% Time-varying vortex strength
Gamma_max = 0.25 * U_exit * D_chamber;
tau_formation = 0.02;

% Storage arrays
```

```
max_frames = min(100, Nt);
U_history = zeros(Ny, Nx, max_frames);
T_history = zeros(Ny, Nx, max_frames);
c_history = zeros(Ny, Nx, max_frames);
Da_history = zeros(1, max_frames);
time_history = zeros(1, max_frames);

frame_count = 1;

%% ==================== 5. ANIMATION SETUP ==============================
fig = figure('Position', [50, 50, 1400, 900], 'Color', 'white', 'Name', 'Pulsejet Vortex Flame Holder');

% Subplot 1: Velocity field - USING pcolor INSTEAD OF imagesc
subplot(3, 4, [1, 2, 5, 6]);
velocity_data = sqrt(U.^2 + V.^2);
h_vel = pcolor(X, Y, velocity_data);
set(h_vel, 'EdgeColor', 'none', 'FaceColor', 'interp');
hold on;
% Create streamlines with proper initialization
starty = linspace(0.01, D_chamber/2-0.01, 10);
startx = 0.05 * ones(size(starty));
% Check if streamline function works
if exist('streamline', 'file')
    h_stream = streamline(X, Y, U', V', startx, starty);
    set(h_stream, 'Color', 'w', 'LineWidth', 1);
end
h_vortex = plot(x_vortex, y_vortex, 'wo', 'MarkerSize', 10, 'LineWidth', 2);
hold off;
axis equal tight;
hc = colorbar;
ylabel(hc, 'Velocity [m/s]');
caxis([0, 80]);
colormap(jet);
title('Velocity Field & Streamlines');
xlabel('Axial Position [m]');
ylabel('Radial Position [m]');

% Subplot 2: Temperature field
subplot(3, 4, [3, 4]);
h_temp = pcolor(X, Y, T);
set(h_temp, 'EdgeColor', 'none', 'FaceColor', 'interp');
axis equal tight;
hc2 = colorbar;
ylabel(hc2, 'Temperature [K]');
caxis([300, 2200]);
colormap(hot);
title('Temperature Field');
xlabel('Axial Position [m]');
ylabel('Radial Position [m]');

% Subplot 3: Progress variable (flame)
subplot(3, 4, 7);
h_flame = pcolor(X, Y, c);
set(h_flame, 'EdgeColor', 'none', 'FaceColor', 'interp');
axis equal tight;
hc3 = colorbar;
ylabel(hc3, 'Progress');
caxis([0, 1]);
colormap(spring);
title('Reaction Progress (c)');
xlabel('Axial Position [m]');
ylabel('Radial Position [m]');

% Subplot 4: Vorticity field
subplot(3, 4, 8);
h_vort = pcolor(X, Y, omega);
set(h_vort, 'EdgeColor', 'none', 'FaceColor', 'interp');
axis equal tight;
hc4 = colorbar;
ylabel(hc4, 'Vorticity [1/s]');
colormap(cool);
title('Vorticity Field');
xlabel('Axial Position [m]');
ylabel('Radial Position [m]');

% Subplot 5: Damköhler number evolution
subplot(3, 4, [9, 10]);
h_da = plot(0, 0, 'b-', 'LineWidth', 2);
hold on;
h_da_target = plot([0, T_total], [1, 1], 'r--', 'LineWidth', 1.5);
hold off;
xlabel('Time [s]');
ylabel('Damköhler Number (Da)');
title('Flame Stability Parameter');
grid on;
xlim([0, T_total]);
```

```
ylim([0, 3]);
legend('Actual Da', 'Stability Threshold (Da=1)', 'Location', 'northwest');

% Subplot 6: Residence time ratio
subplot(3, 4, [11, 12]);
h_residence = plot(0, 0, 'g-', 'LineWidth', 2);
hold on;
h_res_target = plot([0, T_total], [1, 1], 'r--', 'LineWidth', 1.5);
hold off;
xlabel('Time [s]');
ylabel('Residence Time Ratio');
title('τ_residence / τ_chemical');
grid on;
xlim([0, T_total]);
ylim([0, 15]);
legend('Actual Ratio', 'Stability Threshold (=1)', 'Location', 'northwest');

%% ==================== 6. MAIN SIMULATION LOOP ========================
fprintf('Starting simulation...\n');
fprintf('Time steps: %d, dt = %.2e s\n', Nt, dt);
tic;

for n = 1:Nt
    t = n * dt;

    % ========== VORTEX FORMATION ==========
    if t < tau_formation
        Gamma = Gamma_max * (1 - exp(-t/(tau_formation/3)));
    else
        Gamma = Gamma_max * exp(-(t-tau_formation)/(5*tau_formation));
    end

    % Lamb-Oseen vortex velocity field
    r = sqrt((X - x_vortex).^2 + (Y - y_vortex).^2 + 1e-6);
    theta = atan2(Y - y_vortex, X - x_vortex);

    u_theta = (Gamma./(2*pi*r)) .* (1 - exp(-r.^2./(4*nu*(t+0.001))));

    u_vortex = -u_theta .* sin(theta);
    v_vortex = u_theta .* cos(theta);

    omega = (Gamma./(pi*(4*nu*(t+0.001)))) .* exp(-r.^2./(4*nu*(t+0.001)));

    % ========== COMBINE FLOW FIELDS ==========
    U_total = U + u_vortex;
    V_total = V + v_vortex;

    % ========== FLAME PROPAGATION MODEL ==========
    u_rms = sqrt(mean(u_vortex(:).^2));
    S_T = S_L * (1 + (u_rms/S_L)^0.5);

    reaction_rate = A * exp(-Ea./(R_gas*T)) .* c .* (1-c);

    [dc_dx, dc_dy] = gradient(c, dx, dy);
    convection = U_total .* dc_dx + V_total .* dc_dy;
    [d2c_dx2, d2c_dy2] = gradient(dc_dx, dx, dy);
    diffusion = k_thermal/(rho*cp) * (d2c_dx2 + d2c_dy2);

    c = c + dt * (convection + diffusion + reaction_rate);
    c = max(0, min(1, c));

    T = T_fresh + (T_flame - T_fresh) * c;

    % ========== CALCULATE STABILITY PARAMETERS ==========
    vortex_mask = r < R_vortex;
    if any(vortex_mask(:))
        U_vortex_avg = mean(abs(u_vortex(vortex_mask)));
        T_avg_vortex = mean(T(vortex_mask));
    else
        U_vortex_avg = 0;
        T_avg_vortex = T_fresh;
    end

    if U_vortex_avg > 0
        tau_flow = L_vortex / U_vortex_avg;
    else
        tau_flow = 1e6;
    end

    if T_avg_vortex > T_fresh
        tau_chem = 1 / (A * exp(-Ea/(R_gas*T_avg_vortex)));
    else
        tau_chem = 1e6;
    end

    if tau_chem > 0 && tau_chem < 1e5
```

```
        Da = tau_flow / tau_chem;
    else
        Da = 0;
    end

    if U_vortex_avg > 0
        tau_residence = (2*pi*R_vortex) / U_vortex_avg;
        residence_ratio = tau_residence / tau_chem;
    else
        residence_ratio = 0;
    end

    % ========== STORE FOR ANIMATION ==========
    if mod(n, floor(Nt/max_frames)) == 0 && frame_count <= max_frames
        U_history(:, :, frame_count) = sqrt(U_total.^2 + V_total.^2);
        T_history(:, :, frame_count) = T;
        c_history(:, :, frame_count) = c;
        Da_history(frame_count) = Da;
        time_history(frame_count) = t;
        frame_count = frame_count + 1;
    end

    % ========== UPDATE ANIMATION ==========
    if mod(n, floor(Nt/30)) == 0 || n == Nt
        % Update velocity field
        set(h_vel, 'CData', sqrt(U_total.^2 + V_total.^2));
        set(h_vel, 'ZData', sqrt(U_total.^2 + V_total.^2));

        % Update temperature field
        set(h_temp, 'CData', T);
        set(h_temp, 'ZData', T);

        % Update progress variable
        set(h_flame, 'CData', c);
        set(h_flame, 'ZData', c);

        % Update vorticity
        set(h_vort, 'CData', omega);
        set(h_vort, 'ZData', omega);

        % Update vortex position
        set(h_vortex, 'XData', x_vortex, 'YData', y_vortex);

        % Update plots
        if frame_count > 1
            set(h_da, 'XData', time_history(1:frame_count-1), ...
                      'YData', Da_history(1:frame_count-1));

            res_data = residence_ratio * ones(1, frame_count-1);
            set(h_residence, 'XData', time_history(1:frame_count-1), ...
                             'YData', res_data);
        end

        % Update titles
        subplot(3, 4, [1, 2, 5, 6]);
        title_str = sprintf('Velocity Field\nTime: %.3f s, Da: %.2f', t, Da);
        title(title_str);

        subplot(3, 4, [3, 4]);
        title(sprintf('Temperature Field\nMax: %.0f K', max(T(:))));

        % Update annotation
        if ishandle(fig)
            % Remove old annotations
            anns = findall(fig, 'Type', 'annotation');
            delete(anns);

            % Add status annotation
            if Da > 1
                status = 'STABLE';
                color = [0.8, 1, 0.8];
            else
                status = 'UNSTABLE';
                color = [1, 0.8, 0.8];
            end

            annotation('textbox', [0.4, 0.02, 0.2, 0.05], ...
                'String', sprintf('Flame: %s (Da=%.2f)', status, Da), ...
                'FitBoxToText', 'on', 'BackgroundColor', color, ...
                'FontSize', 10, 'FontWeight', 'bold', 'HorizontalAlignment', 'center');
        end

        drawnow;
    end

    % Adaptive time step
```

```
    max_velocity = max(abs(U_total(:)));
    dt_max = 0.8 * min(dx, dy) / (max_velocity + 1e-6);
    if dt > dt_max
        dt = dt_max * 0.9;
    end
end

simulation_time = toc;
fprintf('Simulation completed in %.1f seconds\n\n', simulation_time);

%% ==================== 7. POST-PROCESSING ============================
figure('Position', [100, 100, 1200, 500], 'Name', 'Stability Analysis');

% Plot 1: Damköhler number
subplot(1, 2, 1);
if frame_count > 1
    plot(time_history(1:frame_count-1), Da_history(1:frame_count-1), ...
         'b-', 'LineWidth', 2.5);
end
hold on;
plot([0, T_total], [1, 1], 'r--', 'LineWidth', 2);
if frame_count > 1
    fill([0, time_history(1:frame_count-1), T_total], ...
         [0, Da_history(1:frame_count-1), 0], 'b', 'FaceAlpha', 0.2);
end
hold off;
xlabel('Time [s]', 'FontSize', 12);
ylabel('Damköhler Number (Da)', 'FontSize', 12);
title('Flame Stability Evolution', 'FontSize', 14, 'FontWeight', 'bold');
grid on;
legend('Actual Da', 'Stability Threshold', 'Location', 'best');
xlim([0, T_total]);
if frame_count > 1
    ylim([0, max(Da_history)*1.1]);
end

% Plot 2: Parameters
subplot(1, 2, 2);
if frame_count > 1
    yyaxis left;
    mean_temp = squeeze(mean(mean(T_history(:, :, 1:frame_count-1), 1), 2));
    plot(time_history(1:frame_count-1), mean_temp, 'r-', 'LineWidth', 2);
    ylabel('Mean Temperature [K]', 'FontSize', 12);
    ylim([T_fresh, T_flame]);

    yyaxis right;
    vortex_str = squeeze(mean(mean(U_history(:, :, 1:frame_count-1), 1), 2));
    plot(time_history(1:frame_count-1), vortex_str, 'b-', 'LineWidth', 2);
    ylabel('Vortex Strength [m/s]', 'FontSize', 12);
    ylim([0, max(vortex_str)*1.1]);
end
xlabel('Time [s]', 'FontSize', 12);
title('Stabilization Parameters', 'FontSize', 14, 'FontWeight', 'bold');
grid on;
legend('Temperature', 'Vortex Strength', 'Location', 'best');
xlim([0, T_total]);

%% ==================== 8. FINAL ANALYSIS ================================
fprintf('\n=== FINAL ANALYSIS ===\n');
fprintf('Final Damköhler number: %.3f\n', Da);
if Da > 1
    fprintf('→ FLAME STABLE (Da > 1)\n');
else
    fprintf('→ FLAME UNSTABLE (Da < 1)\n');
end

fprintf('\nVortex Characteristics:\n');
fprintf('  Diameter: %.3f m (%.1f%% of chamber)\n', D_vortex, D_vortex/D_chamber*100);
fprintf('  Circulation: %.2f m²/s\n', Gamma);
fprintf('  Average velocity: %.1f m/s\n', U_vortex_avg);

fprintf('\nTime Scales:\n');
fprintf('  Chemical time: %.2f ms\n', tau_chem*1000);
fprintf('  Flow time: %.2f ms\n', tau_flow*1000);
fprintf('  Residence time: %.2f ms\n', tau_residence*1000);

fprintf('\nDesign Recommendations:\n');
fprintf('  Optimal D_vortex/D_chamber: 0.55-0.65\n');
fprintf('  Optimal location: 30-50%% from head\n');
fprintf('  Required Da > 1.5 for safety margin\n');

%% ==================== 9. SAVE RESULTS ================================
% Save figure
saveas(fig, 'pulsejet_vortex_results.png');
fprintf('\nResults saved as pulsejet_vortex_results.png\n');

```

```
% Save data
save('vortex_simulation_data.mat', 'time_history', 'Da_history', ...
     'U_history', 'T_history', 'c_history', 'frame_count');
fprintf('Data saved as vortex_simulation_data.mat\n');
```